

Claude Code Cookbook

从零上手到 Agent Runtime 工程实践

2026-08-13 Edition

基于 Anthropic / Claude Code 当前官方文档与工程博客整理

如何使用这本 Cookbook

这不是一份“命令大全”，而是一份面向真实开发场景的操作手册。每个 Recipe 都用同一套结构组织：目标、操作步骤、KNOW-WHY、可复制配置、常见坑和完成标准。

核心模型 Claude Code 的效果不是只由模型决定，而更接近 $\text{Model} \times \text{Context} \times \text{Tools} \times \text{Feedback} \times \text{Isolation} \times \text{Policy}$ 。真正的效率提升来自让 Agent 看见正确上下文、获得合适工具、在可控边界内行动，并用测试/构建/评测形成闭环。

层级	核心能力	你应该关注什么
L1 交互	Prompt / Chat	如何给任务边界与验收条件
L2 Agent Loop	Read / Search / Edit / Bash	如何让 Agent 自主探索并验证
L3 Context Engineering	CLAUDE.md / Rules / Memory / Compact	上下文加载、污染与压缩
L4 Runtime	Permissions / Sandbox / Hooks / MCP	能力边界、外部工具、确定性控制
L5 Multi-Agent	Subagents / Worktrees / Agent Teams	隔离、并行、协调与合并

版本说明 本文按 2026-08-13 官方文档整理。Claude Code 迭代很快：Auto mode 仍为 research preview；Agent Teams 仍为 experimental；Agent View 仍为 research preview。涉及实验能力时，优先运行 `claude --version` 并查看当前官方文档。

目录

Part I 心智模型与第一小时

- Recipe 01 安装与环境自检
- Recipe 02 新项目 30 分钟初始化
- Recipe 03 先探索后修改
- Recipe 04 Plan → Implement 双阶段
- Recipe 05 Verification-first Bug Fix

Part II Context / Memory / Session

- Recipe 06 设计一个有效的 CLAUDE.md
- Recipe 07 CLAUDE.md 分层与 CLAUDE.local.md
- Recipe 08 path-scoped Rules
- Recipe 09 Auto Memory 治理
- Recipe 10 `/context /compact /clear /btw`
- Recipe 11 Session / Resume / Branch / Rewind

Part III Permissions 与 Sandbox

- Recipe 12 选择 Permission Mode
- Recipe 13 settings.json Allow / Ask / Deny
- Recipe 14 启用 Sandbox
- Recipe 15 保护 secrets 与高风险路径
- Recipe 16 面向团队的最小安全基线

Part IV Skills 与 Hooks

- Recipe 17 第一个 Skill
- Recipe 18 副作用 Skill: 只允许人工触发
- Recipe 19 Skill + context: fork
- Recipe 20 Progressive Disclosure
- Recipe 21 PostToolUse 自动格式化
- Recipe 22 PreToolUse 保护关键文件
- Recipe 23 Stop Hook 做质量门禁

Part V MCP / Plugins / Code Intelligence

- Recipe 24 接入 HTTP / stdio MCP
- Recipe 25 MCP Scope 与团队共享
- Recipe 26 MCP Tool Search 控制 Context
- Recipe 27 MCP + Skill 组合
- Recipe 28 安装与管理 Plugins
- Recipe 29 自建 Plugin 与 LSP

Part VI Subagents / Worktrees / Multi-Agent

- Recipe 30 创建专用 Subagent
- Recipe 31 用 Subagent 隔离研究 Context
- Recipe 32 Worktree 隔离并行 Session
- Recipe 33 Subagent + Worktree
- Recipe 34 Agent Teams
- Recipe 35 竞争假设调试

Part VII CLI 自动化与长任务

- Recipe 36 claude -p 非交互模式
- Recipe 37 JSON / JSON Schema 结构化输出
- Recipe 38 控制 turns / budget / session persistence
- Recipe 39 长任务的进度文件与增量交付
- Recipe 40 /batch 与大规模机械改造

Part VIII 故障排查与工程模板

- 失败模式诊断
- 命令速查
- settings.json 模板
- CLAUDE.md 模板
- Skills / Agents / Hooks 模板
- Prompt Library
- 官方资料索引

Part I 心智模型与第一小时

Claude Code 是一个会循环调用工具的 agentic coding environment。典型循环可以理解为：读取任务与上下文 → 选择工具 → 执行 → 获得结果 → 重新判断 → 直到没有新的工具调用。复杂任务的质量往往取决于这个循环能否获得有效反馈，而不是某一句 prompt 是否“神奇”。

Agent Loop

```

User Prompt
  ↓
Claude evaluates state
  ↓
Tool calls: Read / Grep / Edit / Bash / Agent / MCP ...
  ↓
Tool results
  ↓
Claude re-evaluates
  ↓
Verification / more tools / final answer
  
```

KNOW-WHY 把 Claude Code 当成“Harness”而不是“聊天框”，你才会自然地设计 Context、工具、权限、验证器和隔离边界。

Recipe 01 安装与环境自检

目标：用当前推荐方式安装 Claude Code，并验证版本、登录与项目启动。

KNOW-WHY Native Install 会自动后台更新；Homebrew / WinGet 更适合由包管理器控制升级。先确认安装方式和版本，能减少“教程正确但本机版本不支持”的问题。

操作步骤

1. macOS / Linux / WSL 优先使用 Native Install；macOS 也可用 Homebrew。
2. Windows 可用 PowerShell 安装脚本或 WinGet；原生 Windows 推荐安装 Git for Windows 以获得 Bash 工具。
3. 运行 `claude --version`，然后进入一个 Git 项目执行 `claude`。
4. 出现异常时优先用 `/doctor` 或安装文档排查。

macOS / Linux / WSL

```

curl -fsSL https://claude.ai/install.sh | bash
claude --version
cd your-project
claude
  
```

macOS Homebrew

```

brew install --cask claude-code
# 升级
brew upgrade claude-code
  
```

Windows PowerShell

```

irm https://claude.ai/install.ps1 | iex
claude --version
  
```

Windows WinGet

```

winget install Anthropic.ClaudeCode
winget upgrade Anthropic.ClaudeCode
  
```

KNOW-HOW 把“版本 + 安装方式 + shell 类型”写进团队故障排查模板。很多 Claude Code 问题其实是环境差异，而不是模型问题。

常见坑

- 不要继续沿用早期 npm 全局安装教程作为默认路径。
- Homebrew / WinGet 不会像 Native Install 一样自动后台更新。

完成标准 `claude --version` 有输出；进入项目运行 `claude` 后能正常登录并读取当前目录。

官方依据：S1

Recipe 02 新项目 30 分钟初始化

目标：第一次进入一个仓库时，建立可重复的项目上下文与安全边界。

KNOW-WHY 第一次会话的目标不是马上写代码，而是降低模型对仓库结构、命令、规则的猜测空间。初始化阶段多花 10 分钟，通常会减少后续大量纠错。

操作步骤

1. 从仓库根目录启动 `claude`。
2. 运行 `/init` 生成或改进 `CLAUDE.md`。
3. 运行 `/memory` 查看已加载的 `memory files` 与 `Auto Memory` 状态。
4. 运行 `/context` 确认 `CLAUDE.md` / `rules` / `skills` / `MCP` 等启动上下文。
5. 运行 `/permissions`，先配置你愿意自动批准的低风险命令。
6. 需要更高自治时运行 `/sandbox`，先建立文件与网络边界。
7. 让 Claude 只做 `codebase overview`，不要修改代码。

第一次会话建议顺序

```
claude
/init
/memory
/context
/permissions
/sandbox
```

仓库理解 Prompt

Give me an overview of this repository.

Explain:

1. architecture and major modules
2. entry points and data flow
3. build / test / lint / typecheck commands
4. important conventions
5. fragile or security-sensitive areas
6. what you are uncertain about

Do not modify anything.

KNOW-HOW 如果团队有自己的 onboarding 文档，把“人类新同学必须知道但代码里看不出来”的内容优先沉淀进 `CLAUDE.md` / `rules` / `skills`。

常见坑

- /init 生成的是起点，不是最终真相；它发现不了团队隐性规则。
- 不要把“读完整仓库”当目标，应该围绕架构与当前任务逐步收缩搜索范围。

完成标准 你能回答：怎么构建、怎么测试、代码入口在哪里、哪些规则必须遵守、哪些目录不能随便动。

官方依据：S5, S8, S10

Recipe 03 先探索后修改

目标：在复杂任务中把“建立代码世界模型”和“写代码”分开。

KNOW-WHY 边读边改容易产生 premature commitment：Agent 看到一个局部线索就开始行动，后续新证据往往会被已有修改路径锚定。读-only exploration 可以先扩大证据面。

操作步骤

1. 切换到 Plan Mode，或明确要求只研究、不修改。
2. 要求 Claude 给出相关文件、调用链、证据与未知项。
3. 让它明确列出“事实 / 假设 / 仍需验证”。
4. 只有当根因或修改面足够清楚后，再进入实现。

探索 Prompt

```
Investigate the authentication flow.

Do not change code.
Return:
- entry points
- request / token / session flow
- relevant files and functions
- current tests
- likely failure points
- evidence for each hypothesis
- unknowns that still need verification
```

KNOW-HOW 对于未知根因的 bug，优先要求“提出多个假设并主动寻找反例”，避免第一个合理解释成为锚点。

常见坑

- “读很多文件”不是高质量探索；高质量探索要输出可验证的调用链和证据。
- 不要让 exploration subagent 返回整段日志或几十个文件全文；主会话只需要高价值结论。

完成标准 进入实现前，你能说明“为什么要改这些文件，而不是别的文件”。

官方依据：S2, S10

Recipe 04 Plan → Implement 双阶段

目标：让计划包含 why、影响面、验证方式，而不是 Todo List。

KNOW-WHY “修改 A、加测试、运行测试”只是任务列表。真正有用的 Plan 应该降低实现阶段的不确定性，并提前暴露不必要的架构变更。

操作步骤

1. 先 /plan 或切到 plan mode。
2. 要求每一步写明 why、文件/函数、复用模式、回归风险和验证命令。
3. 人工检查是否存在越界重构、API 变化、数据迁移。
4. 批准后切回 acceptEdits / default / auto 等合适模式执行。
5. 实现中如偏离 Plan，要求解释原因。

高信息密度 Plan Prompt

```
Create an implementation plan.

For every step include:
- why the change is needed
- exact files / functions involved
- existing patterns to reuse
- behavior before / after
- regression risks
- tests required
- exact verification command

Avoid unrelated refactoring.
Do not implement yet.
```

KNOW-HOW 大型功能可以把“需求访谈 / SPEC”和“实现”拆成两个 session，第二个 session 只携带经过整理的规格与项目规则。

常见坑

- Plan 过细会浪费 token；重点是决策点、风险和验收，不是逐行代码。
- 不要把“计划已写完”当“根因已验证”；对关键假设仍应要求证据。

完成标准 Plan 能让另一个工程师不重新探索仓库就开始实现。

官方依据：S5, S11

Recipe 05 Verification-first Bug Fix

目标：先构造失败信号，再让 Agent 在闭环里修复。

KNOW-WHY 没有验证器的 Agent 是 open-loop：它只能凭语言判断“应该好了”。有测试、构建、lint 或可复现步骤后，Agent 才能根据真实反馈不断纠偏。

操作步骤

1. 让 Claude 先复现问题。
2. 优先写一个会失败的最小测试，或者给出确定的复现命令。
3. 只在失败被观察到以后开始修复。
4. 修复后运行目标测试，再运行相关回归测试。
5. 最后检查 diff 是否有关改动。

Bug Fix Prompt

```
Reproduce this bug with a failing test first.
Do not implement a fix until the failure is observed.
```

Then:

1. identify the root cause
2. implement the minimal fix
3. run the focused test
4. run relevant regression tests
5. run lint/typecheck if applicable
6. review the final diff for unrelated changes

Before finishing, show the verification evidence.

KNOW-HOW 把“Done means”写进 Prompt，比额外堆很多角色扮演和思维指令更有用。

常见坑

- 不要接受“代码看起来正确”作为验收证据。
- 如果测试环境不稳定，先让 Claude 区分产品 bug 与基础设施噪声。

完成标准 修复前有 failing signal；修复后相同 signal 通过；回归测试通过；最终 diff 与根因一致。

官方依据：S5

Part II Context / Memory / Session

Claude Code 的 context window 不只是聊天历史，还包含系统指令、CLAUDE.md、Auto Memory、文件内容、命令输出、已加载 Skills、工具信息等。Context 是稀缺工作内存，不是日志仓库。

Context Engineering 目标不是“给模型更多”，而是让每个 token 对当前决策更有价值。持久规则放 CLAUDE.md；按路径生效的规则放 .claude/rules；任务流程放 Skill；大量探索放 Subagent；短暂旁问用 /btw；长会话用 /compact。

Recipe 06 设计一个有效的 CLAUDE.md

目标：把每次都要重复解释的项目不变量压缩成高价值常驻上下文。

KNOW-WHY CLAUDE.md 每个 session 都会进入上下文，因此它既是“长期项目提示词”，也是持续的 token 成本。官方建议目标控制在 200 行以内，具体、可验证、无冲突。

操作步骤

1. 只记录模型从代码不容易推断的事实。
2. 优先写 build/test/lint/typecheck 命令、架构边界、代码约定、禁止事项。
3. 用标题 + bullets，避免大段 prose。
4. 规则写成可验证动作，例如“提交前运行 npm test”。
5. 每次 Claude 第二次犯同类错误时，考虑把纠正沉淀进 CLAUDE.md。

推荐骨架

```
# Project
- Service: ...
- Language/runtime: ...

# Architecture
- API: src/api/
```



```

- Domain: src/domain/
- Persistence: src/db/

# Commands
- Install: ...
- Test: ...
- Lint: ...
- Typecheck: ...

# Invariants
- Domain layer must not depend on web framework
- Never access DB directly from HTTP handlers

# Workflow
- Reproduce bugs before fixing
- Run focused tests, then regression tests

# Git
- Keep commits focused
- Never force-push unless explicitly asked

```

KNOW-HOW 把 CLAUDE.md 看成 Constitution，而不是 Wiki。百科知识与 SOP 应该按需加载。

常见坑

- 不要写 “write clean code / use best practices” 这类低信息规则。
- 不要把整份 API 文档、发布 SOP、几十步流程全部塞进去。
- 不要同时存在互相冲突的 parent / project / local 规则。

完成标准 你删除任何一行时，都能回答 “删掉后 Claude 更容易犯什么具体错误”。

官方依据：S8

Recipe 07 CLAUDE.md 分层与 CLAUDE.local.md

目标：把组织、个人、项目、局部目录规则放在正确作用域。

KNOW-WHY Claude Code 会沿目录层级加载多个 CLAUDE.md，而不是只读一个根文件。更靠近工作目录的规则在上下文中更晚出现；子目录规则可按需加载。分层比单个巨大文件更容易维护。

操作步骤

1. 组织级规则由管理员放在 managed policy 路径。
2. 个人跨项目偏好放 ~/.claude/CLAUDE.md。
3. 团队共享项目规则放 ./CLAUDE.md 或 ./claude/CLAUDE.md。
4. 个人且项目特定的配置放 ./CLAUDE.local.md，并加入 .gitignore。
5. 大型 monorepo 在子目录放更局部 CLAUDE.md，或使用 path-scoped rules。

常见布局

```

repo/
├── CLAUDE.md
├── CLAUDE.local.md      # gitignored
├── .claude/
│   └── rules/
├── backend/
│   └── CLAUDE.md
└── frontend/

```

```

└── CLAUDE.md

~/ .claude/CLAUDE.md      # personal, all projects

```

KNOW-HOW 如果规则只对某些文件有效，优先 rules + paths，而不是新建更多常驻 CLAUDE.md。

常见坑

- --add-dir 默认只扩展文件访问，不会自动把那个目录变成完整配置根。
- 如果已有 AGENTS.md，可在 CLAUDE.md 用 @AGENTS.md 导入，避免复制两份规则。

完成标准 /context 能看到实际加载的 memory files；团队规则与个人偏好没有混在同一个版本控制文件里。

官方依据：S8

Recipe 08 path-scoped Rules

目标：让规则只在 Claude 真正处理匹配文件时进入上下文。

KNOW-WHY 对于 monorepo，“所有规则始终加载”会产生无关上下文和 instruction competition。path-scoped rules 把上下文装载从 always-on 变成按文件触发。

操作步骤

1. 在 .claude/rules/ 创建主题文件。
2. 需要条件加载时添加 YAML frontmatter 的 paths。
3. 保持一个规则文件只处理一个主题。
4. 用 /context 观察规则是否按预期加载。

API 规则

```

---
paths:
  - "src/api/**/*.ts"
  - "tests/api/**/*.test.ts"
---

# API Rules
- Validate all external inputs
- Use the standard error response format
- Add or update API tests for behavior changes

```

目录结构

```

project/
├── .claude/
│   └── rules/
│       ├── testing.md
│       ├── security.md
│       ├── frontend/react.md
│       └── backend/api.md

```

KNOW-HOW “Always-on invariant → CLAUDE.md；file-local invariant → Rules；task procedure → Skill” 是最实用的三分法。

常见坑

- paths 规则是在 Claude 读取匹配文件时触发，不是每次工具调用都触发。
- 规则路径不要写得过宽，否则仍然接近常驻上下文。

完成标准 打开不同目录文件时，只出现相关领域规则；无关规则不占主上下文。

官方依据：S8, S9

Recipe 09 Auto Memory 治理

目标：让 Claude 跨 session 保存有价值的经验，同时避免记忆文件变成垃圾场。

KNOW-WHY Auto Memory 是 Claude 自己写的经验，不是强制配置。每个 Git repo 的 worktrees 共享同一份 memory；MEMORY.md 只有前 200 行或 25KB 会在会话启动时加载，因此它应保持索引化。

操作步骤

1. 用 `/memory` 查看或开关 Auto Memory。
2. 把长期可复用发现留给 Auto Memory，例如 build 命令、调试洞察、偏好。
3. 定期检查 MEMORY.md 是否存在错误或过期结论。
4. 把详细主题拆到 `debugging.md`、`api-conventions.md` 等 topic files。
5. 真正必须执行的安全要求不要只靠 memory，改用 `permission` / `hook` / `sandbox`。

关闭某项目 Auto Memory

```
{
  "autoMemoryEnabled": false
}
```

存储结构

```
~/.claude/projects/<project>/memory/
├── MEMORY.md
├── debugging.md
├── api-conventions.md
└── ...
```

KNOW-HOW 当你说“以后记住 X”，先判断 X 是团队规则（CLAUDE.md）、Agent 经验（Memory）还是必须强制执行的控制（Hook/Permission）。

常见坑

- Memory 是 context，不是 policy enforcement。
- 错误记忆会跨 session 反复污染决策，所以值得审计。
- worktree 共享同一 repo memory；不要把分支临时事实误写成全局真理。

完成标准 MEMORY.md 是简洁索引；详细信息拆分；没有把一次性事故当永久规则。

官方依据：S8

Recipe 10 /context /compact /clear /btw

目标：主动治理长会话，不让上下文逐渐变成低价值混合物。

KNOW-WHY 上下文变大不只增加成本，还会让早期细节被压缩、无关日志干扰当前决策。不同命令解决的是不同问题：看、压缩、换任务、旁问。

操作步骤

1. `/context`: 检查当前上下文谁在占空间。
2. `/compact [instructions]`: 仍是同一个任务，但历史太长时做语义压缩。
3. `/clear`: 切换到独立任务，保留可恢复的旧 session 但新上下文为空。
4. `/btw`: 问一个只依赖当前上下文的小问题，不进入主历史；它没有工具访问。

有方向的 compact

```
/compact Preserve:
- confirmed root cause
- files changed
- commands that reproduce and verify the bug
- unresolved risks
Drop verbose exploration logs.
```

旁问

```
/btw what was the config file we decided to change?
```

KNOW-HOW `/btw` 与 Subagent 正好相反：`/btw` 看见完整主 context 但无工具；Subagent 有工具但从隔离 context 开始。

常见坑

- 不要在一个 session 里连续做多个无关 issue。
- `/btw` 不能读取新文件或运行工具，它只看当前上下文。
- compaction 不是完美记忆；持久规则仍应放磁盘上的 CLAUDE.md / Memory / Skills。

完成标准 你能明确解释当前会话中每一类上下文为什么还值得存在。

官方依据：S6, S9, S12

Recipe 11 Session / Resume / Branch / Rewind

目标：把会话当作可命名、可分叉、可回滚的“Context Branch”。

KNOW-WHY 长任务需要的不只是 Git 分支，还需要上下文分支。Session 保存的是决策与工具轨迹；Checkpoint 提供会话级撤销；Git 仍负责永久版本历史。

操作步骤

1. 给重要 session 命名，便于以后恢复。
2. `claude --continue` 继续当前目录最近 session；`claude --resume` 打开选择器。
3. 需要尝试不同方向时 fork/branch，而不是把两个方向混在一条历史里。
4. `Esc Esc` 或 `/rewind` 打开 checkpoint 操作。
5. 记住：Bash 修改的文件不一定被 checkpoint 跟踪。

Session CLI

```
claude --continue
claude --resume
claude --resume <name>
claude -n "auth-refactor"
# session 内也可以 /rename
```

回滚

```
/rewind
# 或按 Esc 两次
```

KNOW-HOW 把 session 命名方式和 issue/branch 对齐，例如 auth-1234-refresh-bug，减少上下文检索成本。

常见坑

- Checkpoint 不是 Git：Bash 的 rm/mv/cp 等变化不在同样的文件编辑跟踪范围内。
- 不要在两个终端同时 resume 同一个 session 而不 fork，否则 transcript 可能交错。

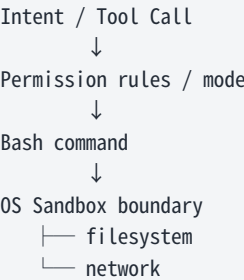
完成标准 你能恢复昨天的任务上下文；能快速尝试替代方案；长期历史仍由 Git 管理。

官方依据：S13, S14

Part III Permissions 与 Sandbox

权限系统回答“Claude 是否被允许请求/执行某能力”，Sandbox 回答“即使执行了，这个进程在 OS 层最多能访问什么”。两者是互补层，而不是二选一。

Defense in depth



Recipe 12 选择 Permission Mode

目标：根据任务风险和自治程度选择合适模式。

KNOW-WHY 权限 prompt 太多会产生 approval fatigue；完全 bypass 又扩大 blast radius。最合适的模式取决于是否在隔离环境、是否需要读-only 规划、是否是无头 CI。

操作步骤

1. 日常复杂变更前先 plan，再切 acceptEdits 或 default。
2. 需要减少确认且当前环境允许时可评估 auto；它仍是 research preview。
3. CI/headless 且工具面固定时考虑 dontAsk + 明确 allowlist。
4. bypassPermissions 只在隔离 VM/container 等环境使用。

KNOW-HOW 推荐默认心智模型：Plan = read-only；acceptEdits = 快速本地编码；dontAsk = 固定 capability surface；Sandbox + Auto = 提升自治但仍保留防御层。

常见坑

- Auto mode 不是 Sandbox 的替代品；它是概率性审批层。
- bypassPermissions 会跳过大部分权限提示，不能在普通个人主机上当默认。

完成标准 你能为每个 mode 解释安全边界和适用场景，而不是只按“少弹窗”选择。

官方依据：S15, S16

Mode	行为	推荐场景
default	标准首次审批	普通交互式工作
acceptEdits	自动接受工作区内文件编辑及部分文件系统命令	日常编码迭代
plan	只读探索与规划	复杂变更前、代码审查
auto	分类器自动审批，research preview	需要较高自治且有额外防线
dontAsk	未预批准的工具直接拒绝	CI、无头 Agent、严格 allowlist
bypassPermissions	跳过权限提示	仅隔离 VM / Container 等受控环境

Recipe 13 settings.json Allow / Ask / Deny

目标：把重复出现的低风险审批沉淀成 declarative policy。

KNOW-WHY 权限规则的价值不是“让 Claude 永远别问”，而是把稳定、可解释的能力边界写进配置。规则按 deny → ask → allow 判断，deny 优先。

操作步骤

- 1. 先用 /permissions 观察实际需要的命令。
- 2. 只 allow 具体且重复出现的安全模式。
- 3. 对 secrets、高风险网络、危险 Git 操作使用 deny 或 ask。
- 4. 项目共享规则放 .claude/settings.json；个人覆盖放 settings.local.json。

保守项目配置示例

```
{
  "permissions": {
    "allow": [
      "Bash(git status)",
      "Bash(git diff *)",
      "Bash(npm test *)",
      "Bash(npm run lint *)"
    ],
    "ask": [
      "Bash(git push *)"
    ],
    "deny": [
      "Read(./env)",
      "Read(./secrets/**)",
      "Edit(./env)",
      "Edit(./secrets/**)"
    ]
  }
}
```

KNOW-HOW Policy 写的是 capability，不是意图。把“允许运行测试”写成窄规则，比“信任 Claude”更可审计。

常见坑

- 不要 allow `Bash(\*)` 这类几乎等价于无限 shell 的规则。
- Bash pattern 不是完整 shell 安全策略；高风险边界应该再叠加 sandbox / hooks。

完成标准 常见低风险命令不再频繁弹窗；敏感资源仍被明确拒绝或要求人工确认。

官方依据：S15, S17

Recipe 14 启用 Sandbox

目标：用 OS 级文件系统与网络隔离降低 Bash 的最大 blast radius。

KNOW-WHY 模型/分类器都是概率性防线；Sandbox 属于环境级确定性边界。即使 Agent 受到 prompt injection 或做出错误判断，子进程仍不能越过配置的 filesystem/network 范围。

操作步骤

1. 在 session 内运行 /sandbox。
2. 个人开发通常先开启 auto-allow sandbox 模式，让 sandbox 内 Bash 少弹窗。
3. 明确允许必要网络域名；不需要的域名不要放开。
4. 如果组织要求强制 sandbox，把 failIfUnavailable 设为 true。
5. 对 Docker 等不兼容命令，谨慎放入 excludedCommands 并保留常规权限流。

Sandbox 基线

```
{
  "sandbox": {
    "enabled": true,
    "autoAllowBashIfSandboxed": true,
    "filesystem": {
      "denyRead": ["~/.ssh", "~/.aws/credentials"]
    },
    "network": {
      "allowedDomains": [
        "github.com",
        "registry.npmjs.org"
      ]
    }
  }
}
```

更严格的组织门禁

```
{
  "sandbox": {
    "enabled": true,
    "failIfUnavailable": true,
    "allowUnsandboxedCommands": false
  }
}
```

KNOW-HOW 安全设计要问的不是“模型会不会犯错”，而是“犯错后最大能造成什么”。

常见坑

- 默认情况下 sandbox 不可用时可以警告后 unsandboxed 运行；严格环境要显式 failIfUnavailable。
- Sandbox 主要约束 Bash 及其子进程；权限规则仍覆盖其他工具。
- 网络只允许真正需要的 host，避免把 `` 作为长期方案。

完成标准 你能明确列出“Agent 即使出错也绝对碰不到的路径/域名”。

官方依据：S18, S19, S20

Recipe 15 保护 secrets 与高风险路径

目标：防止敏感文件进入 Agent 上下文或被工具修改。

KNOW-WHY 外部内容可能包含 prompt injection；本地 secrets 一旦被读入上下文，就可能被错误带入日志、命令或外部工具结果。最优策略是从 capability 层阻止读取，而不是仅告诉模型“不要泄露”。

操作步骤

1. Permission deny 同时覆盖 Read 与 Edit。
2. Sandbox denyRead 限制 Bash 子进程。
3. 不要把 API key 直接写进项目级 .mcp.json；使用环境变量展开或安全认证流。
4. Hooks 脚本也要按不可信输入处理，不要打印 secrets。

Secret deny

```
{
  "permissions": {
    "deny": [
      "Read(./env)",
      "Read(./env.*)",
      "Read(./secrets/**)",
      "Edit(./env)",
      "Edit(./secrets/**)"
    ]
  },
  "sandbox": {
    "filesystem": {
      "denyRead": [
        "~/.ssh",
        "~/.aws/credentials"
      ]
    }
  }
}
```

KNOW-HOW 安全配置最好采用“deny sensitive first，再开放必要能力”的思路。

常见坑

- “CLAUDE.md 写不要读 .env”只是行为提示，不是硬边界。
- MCP/Plugin 也属于外部内容与能力入口，需要同样审计。

完成标准 关键 secrets 既无法被 Read/Edit 工具访问，也无法被 sandboxed Bash 读到。

官方依据：S15, S19

Recipe 16 面向团队的最小安全基线

目标：把个人习惯升级为团队可版本化、可审计的默认安全配置。

KNOW-WHY Agent 能力会随模型升级而提高，依赖每个开发者手工点权限无法形成稳定组织边界。Managed settings、项目 permissions、sandbox 与共享 Skills/Hooks 可以把控制面产品化。

操作步骤

1. 项目提交 .claude/settings.json，定义低风险 allow、敏感 deny。

2. 组织层考虑禁用 `bypassPermissions`，必要时禁用 `auto`。
3. 组织要求 `sandbox` 时 `fail closed`。
4. 限制可用 MCP / plugin marketplace 来源。
5. 把行为规范放 `managed CLAUDE.md`，把技术强制放 `managed settings`。

KNOW-HOW 组织控制面最好区分：Behavioral Guidance (CLAUDE.md) 与 Enforced Capability (settings / sandbox / hooks) 。

常见坑

- CLAUDE.md 不是强制安全策略。
- 不要把团队所有人的个人偏好塞进项目共享配置。

完成标准 新同学 clone repo 后即获得合理默认权限与规则；关键限制不依赖口头培训。

官方依据：S21, S8

Part IV Skills 与 Hooks

Skills 解决“按需加载知识与流程”，Hooks 解决“在生命周期事件上确定执行”。两者最容易混淆，但判断很简单：需要模型理解和灵活应用 → Skill；必须每次发生 → Hook。

Recipe 17 第一个 Skill

目标：把重复 Prompt 变成版本化、可复用的 SOP。

KNOW-WHY Skill 默认只把名称/描述暴露给 Claude，完整内容在被调用时才进入上下文，因此比把长流程常驻 CLAUDE.md 更节省 context。

操作步骤

1. 创建 `.claude/skills/<name>/SKILL.md`。
2. frontmatter 写清 name / description；description 应说明什么时候用。
3. 正文写清输入、步骤、验证与输出格式。
4. 在 session 里用 `/<name>` 手工调用；允许模型调用的 Skill 也可能自动触发。

fix-issue Skill

```
---
name: fix-issue
description: Reproduce and fix a repository bug with tests and minimal changes. Use for issue-driven bug fixes.
argument-hint: "[issue-number-or-description]"
---

Fix: $ARGUMENTS

Workflow:
1. Read the relevant issue/context
2. Reproduce the bug
3. Add a failing test
4. Identify root cause
5. Implement the minimal fix
6. Run focused and regression tests
```

```
7. Run lint/typecheck
8. Review final diff
9. Report root cause and verification evidence
```

KNOW-HOW “A workflow you keep tweaking by hand is a skill that needs another revision.” ——把 Agent workflows 当代码维护。

常见坑

- Skill description 太泛会导致自动调用错误。
- 不要把“只在发布时才需要”的几十页 SOP 写进 CLAUDE.md。

完成标准 同一流程不需要重复粘贴 prompt；团队可以 code review SKILL.md 本身。

官方依据：S3, S22

Recipe 18 副作用 skill：只允许人工触发

目标：避免 Claude 自己决定执行 deploy、commit、发消息等带副作用流程。

KNOW-WHY 可自动匹配的 Skill 很适合知识型能力，但带外部副作用的动作应把触发权留给人。`disable-model-invocation: true` 让它在用户显式调用前不进入 Claude 的可见 skill 描述。

操作步骤

1. 给 deploy / publish / send-message / destructive-maintenance 等 Skill 加 disable-model-invocation。
2. Skill 内仍要先验证前置条件。
3. 高风险实际执行还应依赖 permissions / hooks / external system 权限。

人工触发 deploy

```
---
name: deploy
description: Deploy the application to a requested environment.
disable-model-invocation: true
argument-hint: "[environment]"
---

Deploy $ARGUMENTS:
1. Confirm target environment
2. Run tests
3. Build
4. Show deployment plan
5. Execute only after required permission gates
6. Verify health checks
```

KNOW-HOW 把“什么时候加载流程”和“执行流程需要什么系统权限”视为两层不同控制。

常见坑

- disable-model-invocation 只是控制 Skill 被模型自动调用，不等于系统权限。
- 外部生产环境仍应使用最小权限凭证和独立审批。

完成标准 Claude 不会因为“代码看起来 ready”就自行调用带副作用 Skill。

官方依据：S22

Recipe 19 Skill + context: fork

目标：让一个 workflow 在隔离上下文中执行，避免主会话被大量探索污染。

KNOW-WHY 有些 Skill 本身会读取很多文件或产生大量中间结果。`context: fork` 把 Skill 内容作为隔离 agent 的任务，让主 session 只得到最终结果。

操作步骤

1. 为研究型/审计型 Skill 添加 context: fork。
2. 正文必须包含可执行任务，不要只有静态参考规则。
3. 确保 Skill 的最终输出足够结构化，主会话可以直接消费。

隔离审计 Skill

```
---
name: architecture-audit
description: Audit a subsystem architecture and report coupling, risks, and refactor options.
context: fork
---

Audit $ARGUMENTS.
Read all relevant code, but return only:
- architecture map
- coupling hotspots
- evidence
- top 3 refactor options
- risks and tests
```

KNOW-HOW 隔离不是为了“更聪明”，而是为了控制信息流和主上下文熵。

常见坑

- forked Skill 没有主会话完整历史，所以任务相关上下文要写在调用参数/skill 本身。
- 不要为只包含“编码规范”的 reference Skill 使用 fork。

完成标准 大量文件读取发生在隔离 context；主 context 只保留审计结论。

官方依据：S22, S3

Recipe 20 Progressive Disclosure

目标：把大型领域知识拆成“描述 → SKILL.md → reference/scripts”三级加载。

KNOW-WHY 如果把所有知识都塞入 system/CLAUDE.md，上下文会常驻膨胀。Skill 目录可以把核心 workflow 写短，把参考资料、模板、脚本放旁边，仅在需要时读取。

操作步骤

1. SKILL.md 只保留触发条件、核心流程和资源索引。
2. 把详细 API、例子、模板拆到 reference.md / examples/。
3. 把确定性操作做成 scripts/，让 Agent 调用而不是每次重新生成。
4. description 写清适用场景，帮助 Claude 正确选择。

Skill 目录

```
skills/release/
├── SKILL.md
├── reference.md
├── checklist.md
├── examples/
├── scripts/
│   ├── verify_release.py
│   └── generate_notes.py
```

KNOW-HOW Progressive Disclosure 的核心是“把可能有用的信息从 always-on 改为 demand-driven”。

常见坑

- 不要为了“模块化”拆成几十个微小文件，导致 Agent 不知道该读什么。
- 脚本输出也要控制大小；只返回 Agent 下一步需要的数据。

完成标准 Session 启动只承担低 context 成本；实际需要 release 时才加载完整流程。

官方依据：S3, S22

Recipe 21 PostToolUse 自动格式化

目标：把“每次编辑后格式化”从建议升级为确定性自动化。

KNOW-WHY LLM 指令是概率性的；Hooks 在事件上执行，适合必须发生、可自动验证的机械动作。

操作步骤

1. 先写一个可单独运行、幂等的格式化脚本。
2. 在 settings.json 的 hooks.PostToolUse 中匹配 Edit/Write。
3. 让 hook 只处理被修改文件或相关目录，避免全仓库格式化。
4. 先在小 repo 测试 hook，再共享到团队。

概念配置

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "${CLAUDE_PROJECT_DIR}/.claude/hooks/format-changed.sh"
          }
        ]
      }
    ]
  }
}
```

KNOW-HOW 机械不变量 → Hook；需要语义判断的质量问题 → Skill / Subagent / prompt-based 或 agent hook。

常见坑

- Hook 命令以当前用户权限运行；先审计脚本。
- 不要在 hook 中信任未经验证的文件路径或 shell 输入。

完成标准 无论 Claude 是否“记得”，目标文件都被一致格式化；hook 自身失败时有可诊断输出。

官方依据：S23, S24

Recipe 22 PreToolUse 保护关键文件

目标：在工具实际执行前阻止违反项目 policy 的操作。

KNOW-WHY 只在 CLAUDE.md 写“不要改 migrations”仍是软约束。PreToolUse 可以把关键动作拦在执行前，是更合适的 policy gate。

操作步骤

1. 列出真正需要硬保护的路径/命令。
2. 优先使用 permissions.deny；需要动态判断时再用 PreToolUse。
3. Hook 校验输入、规范化路径、拒绝 path traversal。
4. 对拒绝原因给出清晰反馈，让 Claude 能换方案。

策略思路

```
PreToolUse
  if target is production config -> deny
  if path contains traversal -> deny
  if destructive DB command -> deny / require external approval
  otherwise -> continue
```

KNOW-HOW 能用 declarative deny 完成的，不要先上自定义代码；Hook 用来处理上下文相关、动态 policy。

常见坑

- 不要写一个“黑箱万能安全 hook”；规则越简单越容易审计。
- Hook 本身拥有用户权限，所以它必须比被拦截的命令更可靠。

完成标准 关键资源不能仅靠模型自律访问；拒绝路径可在审计日志中解释。

官方依据：S23, S24, S15

Recipe 23 Stop Hook 做质量门禁

目标：阻止 Agent 在测试、lint、验收未完成时过早宣告结束。

KNOW-WHY 长任务常见失败不是“不会写代码”，而是 premature completion。Stop Hook 可以在结束事件上检查明确的完成条件。

操作步骤

1. 定义最小完成条件：测试通过、无未处理 TODO、特定文件已生成等。
2. 简单条件用 command hook；需要语义检查可评估 prompt/agent hook。
3. Hook 返回不满足原因，让 Agent 继续工作。
4. 避免无限阻塞：条件要客观、可满足。

质量门禁思路

```
Stop event
↓
```

```
check test result / required artifacts
↓
pass -> allow stop
fail -> return reason and continue
```

KNOW-HOW Quality Gate 的关键是把“Done”转换成机器可以观测的状态。

常见坑

- 不要把“代码质量足够好”作为无法客观结束的模糊 gate。
- Agent-based hooks 仍属更实验的能力，生产优先 command hooks。

完成标准 Agent 只有在可验证条件满足后才能结束，而不是仅凭自我判断“完成”。

官方依据：S23

Part V MCP / Plugins / Code Intelligence

MCP 给 Claude 外部能力和数据；Skill 告诉 Claude 如何正确使用这些能力；Plugin 是把 Skills / Agents / Hooks / MCP / LSP 等打包和分发的层。

Recipe 24 接入 HTTP / stdio MCP

目标：把外部服务或本地工具变成 Claude 可调能力。

KNOW-WHY 手工复制数据到 Prompt 既浪费 context，又容易过期。MCP 让 Agent 以工具形式按需获取数据或执行动作。远程服务优先 HTTP；本地进程适合 stdio。

操作步骤

1. 先判断已有成熟 CLI 是否已经足够；没有时再加 MCP。
2. 远程服务使用 `claude mcp add --transport http`。
3. 本地自定义工具使用 stdio。
4. 在 session 用 /mcp 检查连接与认证。
5. 对生产数据优先 read-only / 最小权限。

HTTP

```
claude mcp add --transport http sentry https://mcp.sentry.dev/mcp
# session 内
/mcp
```

stdio

```
claude mcp add --transport stdio db -- <your-server-command> <args>
```

KNOW-HOW 工具设计时返回“Agent 下一步需要的信息”，不要把整个后端对象无差别塞回上下文。

常见坑

- SSE 已被官方标记 deprecated；可用时优先 HTTP。
- 连接成功不代表权限设计正确；工具本身应尽量窄。
- MCP 结果属于外部输入，同样可能包含 prompt injection。

完成标准 Claude 能按需调用外部系统，而不是让用户手工粘贴；连接与认证状态能通过 /mcp 检查。

官方依据: S25, S26

Recipe 25 MCP Scope 与团队共享

目标: 在 local / project / user 作用域之间正确选择。

KNOW-WHY MCP 配置既涉及团队一致性, 也可能包含个人凭证。Scope 决定配置存哪里、谁能看到、哪个定义优先。

操作步骤

1. 个人当前项目实验: local (默认)。
2. 团队共享: project, 写入项目根 .mcp.json 并进入版本控制。
3. 个人跨项目通用工具: user。
4. 共享 config 使用环境变量展开敏感值, 不要提交密钥。

Project scope

```
claude mcp add --transport http paypal --scope project https://mcp.paypal.com/mcp
```

环境变量配置

```
{
  "mcpServers": {
    "api-server": {
      "type": "http",
      "url": "${API_BASE_URL:-https://api.example.com}/mcp",
      "headers": {
        "Authorization": "Bearer ${API_KEY}"
      }
    }
  }
}
```

KNOW-HOW 配置可共享 ≠ 凭证可共享; 把 endpoint/schema 和 secret 生命周期分开。

常见坑

- 同名 server 的优先级是 local > project > user > plugin / connectors。
- 项目 .mcp.json 首次使用会有信任/批准流程, 这是安全边界, 不要绕过。

完成标准 团队共享 capability, 但每个人的 secret 仍由本机环境或 OAuth 管理。

官方依据: S25

Recipe 26 MCP Tool Search 控制 Context

目标: 连接很多 MCP 工具时避免所有 JSON schema 常驻上下文。

KNOW-WHY 当前 Claude Code 默认对 MCP 使用 Tool Search: 启动时只加载工具名称, 真正需要某个工具时再加载 schema。这样工具数量增长不必线性吞噬主 context。

操作步骤

1. 保持 Tool Search 默认开启。
2. 用 /mcp 查看 server 与 token 成本。

3. 只有极少数每回合都需要的核心工具才考虑 `alwaysLoad`。

4. 如果代理/模型不支持 `tool_reference`，再按文档调整 `ENABLE_TOOL_SEARCH`。

阈值模式示例

```
ENABLE_TOOL_SEARCH=auto:5 claude
# 只有当工具 schema 总量小于 context 的约 5% 时才 upfront load
```

Server `alwaysLoad`

```
{
  "mcpServers": {
    "core-tools": {
      "type": "http",
      "url": "https://mcp.example.com/mcp",
      "alwaysLoad": true
    }
  }
}
```

KNOW-HOW 工具规模化的关键是“发现层”和“执行层”分离，这与 Skills 的 `progressive disclosure` 是同一类设计。

常见坑

- 不要为了“确保 Claude 看见”就把所有 server `alwaysLoad`。
- Tool Search 解决 context 成本，不解决工具语义相似导致的选择错误；description 仍要清晰。

完成标准 连接多个 MCP server 后，startup context 仍可控；只有实际使用的 schema 进入主上下文。

官方依据：S25, S3

Recipe 27 MCP + Skill 组合

目标：让“有工具”升级为“知道在业务里怎么正确使用工具”。

KNOW-WHY MCP 提供 capability；Skill 提供组织知识和 SOP。只有 MCP 时，Claude 可能知道“能查数据库”却不知道表语义、优先级、业务约束。

操作步骤

1. MCP 只暴露必要工具。
2. Skill 描述数据模型、业务规则、查询范式。
3. 读写动作分开；敏感写操作人工触发。
4. 用 eval 场景测试 Claude 是否能选对工具、填对参数、解释结果。

组合模式

```
Database MCP
+
analytics-query Skill
↓
Claude knows:
- which tables are authoritative
- which tool to call
- how to filter / aggregate
- what not to modify
```

KNOW-HOW 工具质量应以 Agent Eval 衡量：选错工具、参数错误、结果噪声都属于 Tool Design 问题。

常见坑

- 不要把完整数据库 schema 长期塞 CLAUDE.md。
- Skill 应教“何时/如何用”，而不是复制 MCP tool schema。

完成标准 面对同一个业务问题，Claude 能稳定选择正确 MCP 工具并返回业务可用结果。

官方依据：S3, S26

Recipe 28 安装与管理 Plugins

目标：用 Plugin 复用或分发 Skills / Agents / Hooks / MCP / LSP。

KNOW-WHY 当某套 `.claude/` 配置跨多个仓库复用或需要版本化分发时，继续复制目录会产生漂移。Plugin 是 Claude Code 的 packaging layer。

操作步骤

1. 运行 /plugin 浏览 Installed / Discover。
2. 优先从可信 marketplace 安装。
3. 安装/启用/禁用后可用 /reload-plugins 热加载。
4. 项目级共享插件用 project scope；个人工具用 user/local。
5. 安装前检查插件包含的 hooks、MCP、bin 等能力。

常用操作

```
/plugin
/plugin install <plugin>@<marketplace>
/plugin disable <plugin>@<marketplace>
/plugin enable <plugin>@<marketplace>
/plugin uninstall <plugin>@<marketplace>
/reload-plugins
```

KNOW-HOW 单 repo 快速实验先用 standalone `.claude/`；稳定后再 package 成 plugin。

常见坑

- Plugin 可以包含可执行 hooks、MCP server、bin；不要把“marketplace 中存在”当作安全审计。
- 项目级 plugin 会影响协作者，升级前应测试。

完成标准 一套可复用扩展有明确版本和安装方式，不需要每个 repo 手工复制配置。

官方依据：S27, S28

Recipe 29 自建 Plugin 与 LSP

目标：把组织级编码能力打包，并给 Claude 提供更精确的代码导航。

KNOW-WHY grep/search 能解决很多问题，但 LSP 提供 definition/reference/type diagnostics 等结构化代码智能。Plugin 可以同时携带 Skill、Agent、Hook、MCP 和 LSP 配置。

操作步骤

1. 创建 plugin root 与 .claude-plugin/plugin.json。

- 2. skills/、agents/、hooks/ 等放在 plugin root，不要放进 .claude-plugin/。
- 3. 开发时用 `claude --plugin-dir` 本地加载。
- 4. 需要语言代码智能时优先安装已有官方 LSP plugin；缺失语言再自建 .lsp.json。
- 5. 迭代后通过 marketplace 分发。

Plugin 结构

```
my-plugin/
├── .claude-plugin/
│   └── plugin.json
├── skills/
├── agents/
├── hooks/
├── .mcp.json
├── .lsp.json
└── bin/
```

本地测试

```
claude --plugin-dir ./my-plugin
# 修改后
/reload-plugins
```

KNOW-HOW Plugin 是“分发单元”，不是“上下文单元”；真正进入模型 context 的仍由各组件加载机制决定。

常见坑

- 最常见错误是把 skills/agents/hooks 放在 .claude-plugin/ 目录内部。
- LSP server binary 仍需正确安装/配置；Plugin 不是魔法替代。

完成标准 Plugin 可本地加载；Skill/Agent 出现在界面；LSP 能提供预期诊断/导航。

官方依据：S28, S29

Part VI Subagents / Worktrees / Multi-Agent

并行不是目的。Claude Code 的并行机制主要解决三类不同问题：Context isolation、工作目录/分支 isolation、Agent-to-Agent coordination。选错机制会增加 token 与协调成本。

机制	核心价值	是否独立 Context	是否隔离文件	是否 Agent 间通信
Subagent	委派 + Context 隔离	是	默认否，可 worktree	只回主 Agent
Worktree Session	文件/分支隔离	是（独立 session）	是	否
Agent View	你管理多个后台 session	是	编辑时可工作树隔离	主要对你汇报
Agent Teams	Claude 领导多个独立 teammate	是	默认不自动隔离	是，直接通信

Recipe 30 创建专用 Subagent

目标：把 Security Reviewer、Test Engineer 等角色做成可复用 worker。

KNOW-WHY Subagent 不继承主会话完整历史，拥有独立 context，因此适合 specialized review、搜索、验证等副任务。你还可以限制其 tools、model、turns、memory 等。

操作步骤

1. 在 .claude/agents/ 创建 markdown agent definition。
2. description 写清什么时候使用。
3. tools 只给完成角色需要的能力。
4. Reviewer 尽量默认 read-only。
5. 用 /agents 管理与测试。

security-reviewer.md

```

---
name: security-reviewer
description: Review changed code for security issues and report evidence.
model: sonnet
tools: Read, Grep, Glob, Bash
maxTurns: 20
---

Review for:
- authentication / authorization
- injection
- secret exposure
- unsafe data handling

Return:
1. severity
2. evidence
3. affected file/function
4. recommended remediation

```

KNOW-HOW Generator 与 Evaluator 分离，是降低“自己给自己答案打分”偏差的简单方式。

常见坑

- 不要让 Reviewer 默认拥有 Write/Edit，除非角色真的需要修复。
- 主 Agent 不应把整个对话历史复制给 Subagent；只传任务所需上下文。

完成标准 Subagent 能独立完成 review 并只返回结构化 findings；主 context 不被大量文件读取污染。

官方依据：S4

Recipe 31 用 Subagent 隔离研究 Context

目标：把大量代码搜索、日志分析、文档阅读放到侧边上下文。

KNOW-WHY 很多任务不是模型推理不足，而是主上下文被探索过程淹没。Subagent 可以读几十个文件，但主会话只接收摘要。

操作步骤

1. 主 Agent 给出明确研究问题和输出 schema。
2. Subagent 自己搜索与读取。
3. 要求引用文件/函数作为证据，但不要返回整段文件。
4. 主 Agent 根据 summary 决定下一步是否需要第二个 subagent 验证。

研究委派 Prompt

```
Research the payment retry logic in isolation.
Read as many files as necessary.
Return only:
- execution flow
- source-of-truth files/functions
- retry/backoff semantics
- race conditions you found
- evidence locations
- unanswered questions
```

KNOW-HOW Context isolation 是 Subagent 的第一价值，并行只是第二价值。

常见坑

- 输出 schema 过宽会把大量噪声重新带回主上下文。
- 如果研究结论会直接决定高风险改动，可再派独立 reviewer 交叉验证。

完成标准 主 session 保持短而聚焦；研究中间日志主要留在 subagent context。

官方依据：S3, S4, S9

Recipe 32 Worktree 隔离并行 Session

目标：同时运行多个 Claude session 时避免文件覆盖和 Git 状态冲突。

KNOW-WHY 独立 context 并不能阻止两个 Agent 同时修改同一 checkout。Git worktree 给每个 session 独立工作目录和 branch，同时共享仓库历史。

操作步骤

1. 在 Git 仓库使用 `claude --worktree <name>`。
2. 每个独立任务用独立 worktree。
3. 任务结束后检查 commit/diff，再合并或 cherry-pick。
4. 不要让两个 worktree 任务在逻辑上争抢同一文件所有权。

并行启动

```
# terminal 1
claude --worktree feature-auth

# terminal 2
claude --worktree bug-payment

# short form
claude -w docs-cleanup
```

KNOW-HOW Parallel reasoning 只有叠加 filesystem isolation 后，才真正适合并行软件工程。

常见坑

- Worktree 隔离文件，不自动解决业务冲突；最终 merge 仍可能冲突。
- 一些 gitignored 文件需要额外复制/配置，按官方 .worktreeinclude 机制处理。

完成标准 两个 Claude 可以同时修改而不会互相覆盖 working tree；每个任务有独立 branch/diff。

官方依据：S30

Recipe 33 Subagent + Worktree

目标：让 delegated worker 在临时独立 checkout 中直接写代码。

KNOW-WHY 研究型 Subagent 只需要 context isolation；写代码的 Subagent 还需要 filesystem isolation，否则多个 worker 可能互相覆盖。Agent definition 支持 isolation: worktree。

操作步骤

1. 在需要写代码的专用 agent 上设置 isolation: worktree。
2. 确保任务边界按文件/模块切分。
3. worker 完成后由主 Agent review diff/commit。
4. 无变更的临时 worktree 可自动清理。

worktree-isolated agent

```
---
name: migration-worker
description: Implement an isolated migration unit and its tests.
model: sonnet
tools: Read, Grep, Glob, Edit, Write, Bash
isolation: worktree
maxTurns: 30
---

Implement only the assigned migration unit.
Run focused tests and report changed files plus verification.
```

KNOW-HOW 把“谁负责哪组文件”作为 multi-agent plan 的一等公民。

常见坑

- 不要让多个 worker 同时“拥有”同一个共享核心文件。
- 隔离不能替代最终集成测试。

完成标准 worker 的文件修改不会直接影响主 checkout；主 Agent 可独立 review 与集成。

官方依据：S4, S30

Recipe 34 Agent Teams

目标：让多个独立 Claude Code teammate 共享任务列表并互相通信。

KNOW-WHY Subagents 只能向 caller 汇报；Agent Teams 的 teammate 可以直接彼此消息、共享 task list、自主认领任务，更适合需要协作与互相挑战的复杂并行工作。代价是显著更高 token 与协调开销。

操作步骤

1. 先启用实验环境变量。
2. 从研究/review 任务开始，而不是第一天就并行写核心文件。
3. 通常从 3-5 个 teammate 起步。
4. 为每个 teammate 分配独立职责和文件所有权。
5. 需要高风险实现时可要求 teammate 先 plan，由 lead 批准后再实施。
6. 完成后由 lead 清理 team。

启用

```
{
  "env": {
    "CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS": "1"
  }
}
```

启动 Prompt

Create an agent team to review this feature.

Use three teammates:

- security reviewer
- performance reviewer
- test coverage reviewer

Have them share findings, challenge overlaps, and let the lead synthesize a prioritized report.

KNOW-HOW Multi-Agent 的收益来自“独立证据 + 多样视角 + 并行”，不是把同一个 prompt 复制 N 次。

常见坑

- Agent Teams 仍 experimental；会话恢复、task 状态、shutdown 等有已知限制。
- Teams 默认不会像 worktree 一样自动给每个 teammate 隔离文件；写代码时要避免 file conflicts。
- 不要为了“看起来高级”在顺序任务上使用 team。

完成标准 每个 teammate 有独立问题域；没有重复读同一切面；Lead 能综合而不是简单拼接结果。

官方依据：S31, S32

Recipe 35 竞争假设调试

目标：用多个独立 Agent 主动证伪彼此的根因假设。

KNOW-WHY 单 Agent 调试容易在发现第一个合理解释后停止搜索。并行独立假设 + adversarial falsification 可以降低 anchoring。Anthropic 官方 Agent Teams 文档把这作为典型用例。

操作步骤

1. 让 Lead 明确生成多个互斥或竞争 root-cause hypotheses。
2. 每个 teammate 负责一个假设，但必须尝试证伪自己的假设。
3. 让 teammate 阅读彼此结论并主动寻找反证。
4. 最终共识必须包含“哪些实验排除了哪些假设”。

调试 Prompt

Users report the process exits after one message.

Create 4-5 competing root-cause hypotheses.

Assign one teammate per hypothesis.

Each teammate must:

- find supporting evidence
- actively try to disprove its own hypothesis
- challenge at least one other hypothesis

Do not converge until experiments eliminate alternatives.

Return the surviving explanation and reproduction evidence.

KNOW-HOW 这是把科学方法中的“竞争理论 + 可证伪性”引入 Agent 调试。

常见坑

- 不要让所有 teammate 使用同一个宽泛任务，否则很可能得到重复搜索。
- 最终结论必须由实验/代码证据收敛，而不是“多数投票”。

完成标准 报告明确列出被排除假设、证据和最终复现；不是只说“大家都认为 X”。

官方依据：S31

Part VII CLI 自动化与长任务

Claude Code 既是交互式工具，也可以作为 Unix-style command 在脚本、CI 和批处理中使用。进入自动化后，最重要的新增约束是：输出必须可解析、权限必须 fail closed、成本/turns 必须有上限、状态必须可恢复。

Recipe 36 claude -p 非交互模式

目标：把一次 Agent 任务嵌入 shell pipeline 或 CI。

KNOW-WHY 交互式 session 依赖人类随时纠偏；print mode 则必须在 prompt、工具权限和停止条件上更加显式。它适合 review、分析、批量转换、自动化检查。

操作步骤

1. 使用 `claude -p "prompt"`。
2. 需要机器消费时选择 json / stream-json。
3. 只提供任务必要工具和目录。
4. 自动化中把失败 exit code 当作 pipeline 信号处理。

一次性分析

```
claude -p "Summarize the architecture and list risky coupling points."
```

Shell pipeline

```
git diff main --name-only | \
  claude -p "Review these changed files for security and test gaps"
```

JSON output

```
claude -p "List all API endpoints" --output-format json
```

KNOW-HOW 交互模式优化“协作”；print mode 优化“契约”。

常见坑

- 不要把需要多轮人类澄清的需求直接塞进无人值守 CI。
- 自动化 prompt 必须明确失败条件，不要默认“总会成功”。

完成标准 命令可重复运行、输出可被下游消费、失败有明确状态。

官方依据：S1, S33

Recipe 37 JSON / JSON Schema 结构化输出

目标：让 Agent 的最终结果成为稳定机器接口。

KNOW-WHY 仅要求“请输出 JSON”仍可能得到字段漂移。CLI 支持输出格式和 JSON Schema 验证，更适合 CI / scripts。

操作步骤

1. 定义最小 schema，只包含下游真正需要的字段。
2. 使用 --json-schema 让最终结果匹配 schema。
3. 把 verbose reasoning / logs 留给工具输出，不要塞进最终 JSON。
4. 下游仍验证类型与业务约束。

Schema 示例

```
claude -p \
  --json-schema '{
    "type":"object",
    "properties":{"
      "risk":{"type":"string"},
      "files":{"type":"array","items":{"type":"string"}},
      "summary":{"type":"string"}
    },
    "required":["risk","files","summary"]
  }' \
  "Review the current diff"
```

KNOW-HOW 把 Agent 输出看作 API contract，而不是“更漂亮的回答”。

常见坑

- 结构正确不代表事实正确；仍需要 evidence / test。
- Schema 不要一开始设计成巨大通用对象。

完成标准 下游脚本不需要解析自然语言；字段稳定且 schema 校验通过。

官方依据：S33

Recipe 38 控制 turns / budget / session persistence

目标：给无人值守任务明确资源上限和状态策略。

KNOW-WHY Agent loop 可以持续调用工具。自动化环境中如果没有 turns/cost/permission 边界，一个错误目标可能造成无限尝试或意外成本。

操作步骤

1. 用 --max-turns 限制 agentic turns。
2. API 计费场景可用 --max-budget-usd。
3. 不需要恢复的临时任务使用 --no-session-persistence。
4. 需要跨步骤恢复则保留 session id / named session。
5. 结合 dontAsk + allowlist 或受控 sandbox。

受限自动化

```
claude -p \
  --max-turns 8 \
  --max-budget-usd 3.00 \
```



```
--output-format json \
"Analyze the repository migration risks"
```

KNOW-HOW 无人值守 Agent 需要“预算、权限、时间/turn、输出 schema、重试”五类 guardrails。

常见坑

- 低 max-turns 可能把正常复杂任务截断；先用历史数据校准。
- 预算上限不是安全边界，仍需 permissions/sandbox。

完成标准 任务超出预设资源时 fail predictably；不会无限跑。

官方依据：S33

Recipe 39 任的度文件增量交付

目标：让跨多个 context/session 的任务持续推进，而不是每次重新理解。

KNOW-WHY Compaction 能延长单 session，但长周期项目还需要磁盘上的外部状态。Anthropic 的 long-running harness 实践强调 initializer、feature list、progress notes、git history 与可重复启动脚本。

操作步骤

1. 第一次 session 创建明确 feature/task list，所有项默认未完成。
2. 建立 progress.md，记录已完成、当前状态、下一步、已知问题。
3. 创建 init/test 脚本，让新 session 能快速复现环境。
4. 每个 session 只推进少量高优先任务，并留下 commit + progress update。
5. 每次开始先运行 smoke test，确认上一 session 没留下破坏状态。

推荐长任务文件

```
project/
├── CLAUDE.md
├── progress.md
├── feature-list.json
├── scripts/
│   ├── init.sh
│   └── smoke-test.sh
└── ...
```

Session 开始 Prompt

```
Before coding:
1. read progress.md and recent git log
2. run the smoke test
3. choose ONE highest-priority incomplete item
4. implement and verify it
5. commit the working state
6. update progress.md with evidence and next step
```

KNOW-HOW 长时程能力不是“让一个 context 更长”，而是设计可靠的跨 context state handoff。

常见坑

- 不要让长任务 Agent 每个 session 都试图“一次性完成整个产品”。
- progress 文件应该记录状态与证据，而不是复制聊天记录。

完成标准 新的 session 只用很少 token 就能恢复项目状态，并继续一小步可验证进展。

官方依据：S34

Recipe 40 /batch 与大规模机械改造

目标：把 repo-wide 迁移拆成多个 worktree-isolated unit，而不是单 Agent 串行扫全仓库。

KNOW-WHY 当前 Claude Code 提供 /batch 作为大规模机械变更的并行方式：适合能够规划成多个相对独立 unit 的迁移/重构。它不是所有大型功能的默认答案。

操作步骤

- 1. 先用 plan 把变更拆成独立单元。
- 2. 只有单元之间文件重叠较少时使用 /batch。
- 3. 每个 unit 必须有明确测试/验证命令。
- 4. 最终合并后运行全局 integration/regression。

适合 /batch 的任务

Good:

- rename an API pattern across many independent packages
- migrate repeated config format
- mechanical test framework conversion

Poor:

- one tightly coupled core algorithm
- changes where every task edits the same central file

KNOW-HOW 并行化前先问“任务是否可分解”，而不是“能不能多开几个 Agent”。

常见坑

- 并行会放大 token 与集成成本；机械重复任务才最容易获益。
- 每个子任务通过 ≠ 合并后系统通过。

完成标准 并行 unit 各自可验证，最终集成测试通过，冲突可控。

官方依据：S32, S10

Part VIII 故障排查与工程模板

失模式

症状	根因模式	优先修复
Claude 总在改错文件	任务搜索空间太宽 / context 不足	先 Explore；给相关目录；要求 evidence
越改越大	缺少 scope / minimal-change 约束	Plan 中写 non-goals；最终 diff review
说“完成”但没跑测试	没有 verifier / stopping condition	Done means + tests + Stop Hook
对话越来越笨	context 污染 / 过长	/context → /compact 或 /clear；研究用 Subagent
权限弹窗太多	能力边界没有沉淀	窄 allow rules + Sandbox；必要时评估

症状	根因模式	优先修复
		Auto
CLAUDE.md 不听	文件过长/冲突/太泛	<200 行、具体可验证、拆 Rules/Skills
MCP 工具选错	描述重叠 / tool design 差	优化 description；Skill 教业务用法；做 eval
并行 Agent 互相覆盖	没有 filesystem ownership	Worktree / 按文件切分
Agent Team 很贵但没更快	任务不可并行 / 角色重复	减少 teammate；改用 Subagent 或单 session
长任务每次重头来	状态只在聊天里	progress + feature list + commit + init/smoke scripts

命令速查

命令/快捷键	用途
claude	在当前目录启动交互 session
claude --version	检查版本
/init	初始化/改进 CLAUDE.md
/memory	管理 CLAUDE.md 与 Auto Memory
/context	查看 context 消耗
/compact [focus]	压缩当前会话
/clear	开始空上下文新会话
/btw	不写入主历史的旁问；无工具
/plan	切换/进入规划流程
Shift+Tab	循环 Permission Mode
/permissions	管理 allow/ask/deny
/sandbox	配置 Bash sandbox
/mcp	管理 MCP 连接/认证
/agents	管理 Subagents
/tasks	查看后台任务
/diff	查看当前变更
/rewind 或 Esc Esc	回滚/压缩到 checkpoint
/resume	切换/恢复 session
/plugin	管理 plugins/marketplaces
/reload-plugins	热加载 plugin 变化
claude --worktree NAME	启动隔离 worktree session
claude agents	Agent View，管理后台 sessions（research preview）
claude -p "..."	非交互 print mode

推荐 .claude/ 目录

```

project/
├── CLAUDE.md
├── CLAUDE.local.md           # gitignored
├── .mcp.json                 # only if team-shared MCP config is needed
├── .claude/
│   ├── settings.json
│   ├── settings.local.json  # gitignored
│   ├── rules/
│   │   ├── testing.md
│   │   ├── security.md
│   │   ├── backend/
│   │   └── frontend/
│   ├── skills/
│   │   ├── fix-issue/SKILL.md
│   │   ├── review-pr/SKILL.md
│   │   └── release/SKILL.md
│   ├── agents/
│   │   ├── security-reviewer.md
│   │   ├── test-engineer.md
│   │   └── architecture-reviewer.md
│   └── hooks/
│       ├── format-changed.sh
│       ├── verify.sh
│       └── protect-paths.sh

```

职责分层 CLAUDE.md = always-on invariants; Rules = path-scoped invariants; Skills = on-demand SOP / knowledge; Agents = role + context isolation; Hooks = deterministic lifecycle automation; MCP = external capability; Plugins = packaging/distribution; Sandbox/Permissions = runtime boundary。

settings.json 模板：日常开发安全基线

```

{
  "permissions": {
    "allow": [
      "Bash(git status)",
      "Bash(git diff *)",
      "Bash(npm test *)",
      "Bash(npm run lint *)"
    ],
    "ask": [
      "Bash(git push *)"
    ],
    "deny": [
      "Read(./env)",
      "Read(./env.*)",
      "Read(./secrets/**)",
      "Edit(./env)",
      "Edit(./secrets/**)"
    ]
  },
  "sandbox": {
    "enabled": true,
    "autoAllowBashIfSandboxed": true,
    "filesystem": {
      "denyRead": [
        "~/.ssh",
        "~/.aws/credentials"
      ]
    }
  }
}

```

```

    }
  }
}

```

CLAUDE.md 模板：高信息密度版

```

# Project

- Purpose: <one sentence>
- Runtime: <language / version>
- Package manager: <...>

# Architecture

- Entry points: ...
- API layer: ...
- Domain layer: ...
- Persistence: ...
- Tests: ...

# Commands

- Install: `...`
- Dev: `...`
- Focused test: `...`
- Full test: `...`
- Lint: `...`
- Typecheck: `...`
- Build: `...`

# Invariants

- <specific, verifiable rule>
- <specific, verifiable rule>

# Workflow

- Investigate before editing when the root cause is unclear.
- Reproduce bugs before fixing.
- Prefer minimal changes and existing patterns.
- Run focused tests, then relevant regressions.
- Report deviations from an approved plan.

# Git

- Keep commits focused.
- Do not force-push unless explicitly requested.
- Do not commit generated/secrets files.

```

Skill 模板

```

---
name: <skill-name>
description: <what it does + when Claude should use it>
argument-hint: "[input]"
# disable-model-invocation: true    # for side-effect workflows
# context: fork                     # for isolated research workflows
---

Task: $ARGUMENTS

## Preconditions

```

```
- ...

## Workflow
1. ...
2. ...
3. Verify ...

## Do not
- ...

## Output
Return:
- result
- evidence
- verification
- unresolved risks
```

Subagent 模板

```
---
name: <agent-name>
description: <specialization and when to invoke>
model: sonnet
tools: Read, Grep, Glob, Bash
maxTurns: 20
# isolation: worktree
# memory: project
---

You are responsible for <one role>.

Rules:
- stay inside assigned scope
- cite evidence locations
- distinguish fact from hypothesis
- do not modify files unless the role requires it

Return:
1. findings
2. evidence
3. risks
4. recommended next action
```

Prompt Library

下面这些模板可以直接复制到 Claude Code，重点是 Goal / Context / Scope / Process / Constraints / Done means / Evidence。

通用工程任务

```
Goal:
<what must change>

Context:
<why / observed symptom / business behavior>

Relevant areas:
- <paths>

Process:
1. investigate first
2. reproduce / establish baseline
```

- 3. identify root cause
- 4. implement the minimal change
- 5. verify with exact commands
- 6. review final diff

Constraints:

- no unrelated refactoring
- preserve public API unless necessary
- reuse existing patterns

Done means:

- <observable criterion>
- <test/build criterion>
- <no-regression criterion>

Before finishing:

Show root cause, files changed, commands run, results, and unresolved risks.

独立 Review

Review this diff independently.
Do not rewrite code first.

Check:

- correctness / edge cases
- security
- performance regressions
- error handling
- test gaps
- unnecessary scope expansion

For each issue return:

severity | file:line | evidence | why it matters | recommended change

If no issue is supported by evidence, say so.

反锚定调试

Investigate this bug using competing hypotheses.

First create 3-5 plausible root-cause hypotheses.
For each hypothesis:

- supporting evidence to seek
- evidence that would falsify it
- cheapest experiment

Run the experiments before choosing a winner.
Do not fix the bug until the root cause is supported by evidence.

把 Cookbook 沉淀成你自己的 Agent Runtime

如果你的目标不只是“会用 Claude Code”，而是理解和复刻 Agent Harness，可以把学习路径映射为一组逐步可实现的 runtime 组件：

阶段	可实现模块	关键 know-how
s01	Agent Loop	tool result 回填，直到 stop
s02	Tool Runtime	schema、dispatch、路径安全
s03	Permission System	deny/ask/allow、approval flow
s04	Context Manager	history、tool outputs、compaction
s05	Memory	persistent project/user experience
s06	Skills	progressive disclosure、workflow

阶段	可实现模块	关键 know-how
		packaging
s07	Hooks	event lifecycle、deterministic control
s08	Subagents	isolated context + delegation
s09	Sandbox	filesystem/network containment
s10	Multi-Agent	worktree isolation、task graph、messaging

最终目标 从“Prompt Engineering”升级到“Agent System Engineering”：设计模型每一轮能看到什么、能做什么、何时必须停、如何验证、如何留下状态、出错时最大影响范围是什么。

官方资料索引

以下为本 Cookbook 的主要官方依据。访问日期：2026-08-13。Claude Code 更新频繁，涉及命令、实验状态与配置字段时请以当前官方文档为准。

- S1 Claude Code overview - <https://code.claude.com/docs/en/overview>
- S2 How Claude Code works - <https://code.claude.com/docs/en/how-claude-code-works>
- S3 Extend Claude Code - <https://code.claude.com/docs/en/features-overview>
- S4 Create custom subagents - <https://code.claude.com/docs/en/sub-agents>
- S5 Best Practices for Claude Code - <https://code.claude.com/docs/en/best-practices>
- S6 Commands - <https://code.claude.com/docs/en/commands>
- S7 Interactive mode - <https://code.claude.com/docs/en/interactive-mode>
- S8 How Claude remembers your project - <https://code.claude.com/docs/en/memory>
- S9 Explore the context window - <https://code.claude.com/docs/en/context-window>
- S10 Common workflows - <https://code.claude.com/docs/en/common-workflows>
- S11 Effective context engineering for AI agents - <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>
- S12 Manage sessions - <https://code.claude.com/docs/en/sessions>
- S13 Checkpointing - <https://code.claude.com/docs/en/checkpointing>
- S14 CLI reference - <https://code.claude.com/docs/en/cli-usage>
- S15 Configure permissions - <https://code.claude.com/docs/en/permissions>
- S16 Claude Code auto mode: a safer way to skip permissions - <https://www.anthropic.com/engineering/claude-code-auto-mode>
- S17 Claude Code settings - <https://code.claude.com/docs/en/settings>
- S18 Sandboxing - <https://code.claude.com/docs/en/sandboxing>
- S19 Beyond permission prompts: Claude Code sandboxing - <https://www.anthropic.com/engineering/claude-code-sandboxing>
- S20 How we contain Claude across products - <https://www.anthropic.com/engineering/how-we-contain-claude>

- S21 Set up Claude Code for your organization - <https://code.claude.com/docs/en/admin-setup>
- S22 Extend Claude with skills - <https://code.claude.com/docs/en/slash-commands>
- S23 Automate workflows with hooks - <https://code.claude.com/docs/en/hooks-guide>
- S24 Hooks reference - <https://code.claude.com/docs/en/hooks>
- S25 Connect Claude Code to tools via MCP - <https://code.claude.com/docs/en/mcp>
- S26 Writing effective tools for AI agents - <https://www.anthropic.com/engineering/writing-tools-for-agents>
- S27 Discover and install prebuilt plugins - <https://code.claude.com/docs/en/discover-plugins>
- S28 Create plugins - <https://code.claude.com/docs/en/plugins>
- S29 Plugins reference - <https://code.claude.com/docs/en/plugins-reference>
- S30 Run parallel sessions with worktrees - <https://code.claude.com/docs/en/worktrees>
- S31 Orchestrate teams of Claude Code sessions - <https://code.claude.com/docs/en/agent-teams>
- S32 Run agents in parallel - <https://code.claude.com/docs/en/agents>
- S33 CLI reference - <https://code.claude.com/docs/en/cli-usage>
- S34 Effective harnesses for long-running agents - <https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents>

最后的工程检查清单

- 任务是否明确了 Goal / Scope / Constraints / Done means?
- 复杂任务是否先 Explore / Plan，而不是边猜边改？
- 是否存在机器可观测的 verifier：test / build / lint / screenshot / benchmark？
- CLAUDE.md 是否短、具体、无冲突，且只保留 always-on invariants？
- 局部规则是否迁移到 .claude/rules/ + paths？
- 重复 SOP 是否变成 Skill，而不是不断复制 Prompt？
- 必须发生的机械动作是否变成 Hook？
- 外部能力是否用最小权限 MCP/CLI，并控制返回 context？
- 并行研究是否用 Subagent 隔离上下文？
- 并行写代码是否用 Worktree / 文件所有权隔离？
- 高自治是否建立 Permissions + Sandbox，而不是盲目 bypass？
- 长任务是否把进度状态写到磁盘并留下可复现命令？

一句话总结 把 Claude Code 用好，关键不是让模型“想得更多”，而是把软件工程里的边界、反馈、状态、验证、职责分离，转换成 Agent 能实际执行的 Harness。